

```
# Lesson 9 - Practical Application Guide
# Authentication & Authorization Practice
```

```
## 🚀 How to Run the Project
```

```
```bash
cd D:\learnlaravel2025\lessons\lesson-09\practice-app
php artisan serve
```
```

Server will run on: `http://localhost:8000`

---

```
## 📋 Prerequisites
```

Before starting, ensure you have:

1. Laravel Breeze installed
2. Database configured in `.env`
3. Migrations run
4. Node.js and NPM installed

```
### Installation Steps:
```

```
```bash
# Navigate to practice-app
cd D:\learnlaravel2025\lessons\lesson-09\practice-app
```

```
# Install Laravel Breeze
composer require laravel/breeze --dev
```

```
# Install Breeze scaffolding
php artisan breeze:install blade
```

```
# Install dependencies
npm install && npm run dev
```

```
# Run migrations
php artisan migrate
```
```

---

```
## ✅ Implemented Exercises
```

### ### Exercise 1: Laravel Breeze Setup

**\*\*Task\*\*:** Install and configure Laravel Breeze authentication system.

**\*\*Files Generated\*\*:**

- `app/Http/Controllers/Auth/` - Authentication controllers
- `resources/views/auth/` - Login, register, reset password views
- `routes/auth.php` - Authentication routes

**\*\*Test Routes\*\*:**

- `/register` - Register new user
- `/login` - Login
- `/dashboard` - Dashboard (requires authentication)
- `/profile` - Edit profile

---

### ### Exercise 2: Creating a Simple Gate

**\*\*File\*\*:** `app/Providers/AppServiceProvider.php`

```php

```
use Illuminate\Support\Facades\Gate;
use App\Models\User;
use App\Models\Post;
```

```
public function boot(): void
```

```
{
```

```
    // Gate: User can update own post
```

```
    Gate::define('update-post', function (User $user, Post $post) {
        return $user->id === $post->user_id;
    });
```

```
    // Gate: Admin access
```

```
    Gate::define('access-admin', function (User $user) {
        return $user->role === 'admin';
    });
```

```
    // Gate: Super admin can do everything
```

```
    Gate::before(function (User $user, string $ability) {
        if ($user->email === 'admin@example.com') {
            return true;
        }
    });
```

```
}
```

```

**\*\*Usage in Controller\*\*:**

```
```php
public function update(Request $request, Post $post)
{
    // Method 1: Check manually
    if (Gate::denies('update-post', $post)) {
        abort(403);
    }

    // Method 2: Authorize (throws 403 automatically)
    Gate::authorize('update-post', $post);

    $post->update($request->validated());
    return redirect()->route('posts.show', $post);
}
```
```

**\*\*Usage in Blade\*\*:**

```
```blade
@can('update-post', $post)
    Edit Post
@endcan

@can('access-admin')
    Admin Panel
@endcan
```
```

---

### ### Exercise 3: Creating a Post Policy

**\*\*Command\*\*:**

```
```bash
php artisan make:policy PostPolicy --model=Post
```
```

**\*\*File\*\*:** `app/Policies/PostPolicy.php`

```
```php
is_published) {
    return true;
}

// Only owner can see drafts
return $user && $user->id === $post->user_id;
}
```

```

/**
 * Only verified users can create posts
 */
public function create(User $user): bool
{
    return $user->email_verified_at !== null;
}

/**
 * Only post owner can update
 */
public function update(User $user, Post $post): bool
{
    return $user->id === $post->user_id;
}

/**
 * Owner or admin can delete
 */
public function delete(User $user, Post $post): bool
{
    return $user->id === $post->user_id
        || $user->role === 'admin';
}

/**
 * Only admin can force delete
 */
public function forceDelete(User $user, Post $post): bool
{
    return $user->role === 'admin';
}
}
...

**Usage in Controller**:
```php
class PostController extends Controller
{
    public function __construct()
    {
        $this->middleware('auth')->except(['index', 'show']);
    }

    public function create()
    {

```

```

        $this->authorize('create', Post::class);
        return view('posts.create');
    }

    public function edit(Post $post)
    {
        $this->authorize('update', $post);
        return view('posts.edit', compact('post'));
    }

    public function destroy(Post $post)
    {
        $this->authorize('delete', $post);
        $post->delete();
        return redirect()->route('posts.index');
    }
}
...

```

---

### ### Exercise 4: Custom Middleware (Admin Only)

**\*\*Create Middleware\*\*:**

```

```bash
php artisan make:middleware EnsureUserIsAdmin
...

```

**\*\*File\*\*:** `app/Http/Middleware/EnsureUserIsAdmin.php`

```

```php
check() {
    return redirect('/login');
}

if (auth()->user()->role !== 'admin') {
    abort(403, 'Unauthorized action');
}

return $next($request);
}
}
...

```

**\*\*Register Middleware\*\*** in `bootstrap/app.php`:

```

```php

```

```

->withMiddleware(function (Middleware $middleware) {
    $middleware->alias([
        'admin' => \App\Http\Middleware\EnsureUserIsAdmin::class,
    ]);
})
...

```

**\*\*Usage in Routes\*\*:**

```

```php
Route::middleware(['auth', 'admin'])->group(function () {
    Route::get('/admin/dashboard', [AdminController::class, 'index']);
    Route::get('/admin/users', [AdminController::class, 'users']);
});
...

```

---

**### Exercise 5: API Authentication with Sanctum**

**\*\*Install Sanctum\*\*:**

```

```bash
composer require laravel/sanctum
php artisan vendor:publish --provider="Laravel\Sanctum\SanctumServiceProvider"
php artisan migrate
...

```

**\*\*Add to User Model\*\*:**

```

```php
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable;
}
...

```

**\*\*API Controller\*\*** (`app/Http/Controllers/Api/AuthController.php`):

```

```php
validate([
    'name' => 'required|string|max:255',
    'email' => 'required|email|unique:users',
    'password' => 'required|string|min:8|confirmed',
]);

$user = User::create([
    'name' => $validated['name'],
    'email' => $validated['email'],

```

```

        'password' => Hash::make($validated['password']),
    ]);

    $token = $user->createToken('auth-token')->plainTextToken;

    return response()->json([
        'user' => $user,
        'token' => $token,
    ], 201);
}

public function login(Request $request)
{
    $credentials = $request->validate([
        'email' => 'required|email',
        'password' => 'required',
    ]);

    if (!Auth::attempt($credentials)) {
        return response()->json([
            'message' => 'Invalid credentials'
        ], 401);
    }

    $user = User::where('email', $request->email)->first();
    $token = $user->createToken('auth-token')->plainTextToken;

    return response()->json([
        'user' => $user,
        'token' => $token,
    ]);
}

public function logout(Request $request)
{
    $request->user()->currentAccessToken()->delete();

    return response()->json([
        'message' => 'Logged out successfully'
    ]);
}

public function me(Request $request)
{
    return response()->json($request->user());
}
}

```

```
...
```

```
**API Routes** (`routes/api.php`):
```

```
```php
```

```
use App\Http\Controllers\Api\AuthController;
```

```
Route::post('/register', [AuthController::class, 'register']);
```

```
Route::post('/login', [AuthController::class, 'login']);
```

```
Route::middleware('auth:sanctum')->group(function () {
```

```
    Route::post('/logout', [AuthController::class, 'logout']);
```

```
    Route::get('/me', [AuthController::class, 'me']);
```

```
});
```

```
...
```

```
**Testing with cURL or Postman**:
```

```
```bash
```

```
# Register
```

```
curl -X POST http://localhost:8000/api/register \
```

```
  -H "Content-Type: application/json" \
```

```
  -H "Accept: application/json" \
```

```
  -d '{"name":"John Doe","email":"john@example.com","password":"password123","password_conf'
```

```
# Login
```

```
curl -X POST http://localhost:8000/api/login \
```

```
  -H "Content-Type: application/json" \
```

```
  -H "Accept: application/json" \
```

```
  -d '{"email":"john@example.com","password":"password123"}'
```

```
# Get User (use token from login response)
```

```
curl -X GET http://localhost:8000/api/me \
```

```
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
```

```
  -H "Accept: application/json"
```

```
...
```

```
---
```

```
### Exercise 6: Roles & Permissions System
```

```
**Add role column to users table** (migration):
```

```
```php
```

```
Schema::table('users', function (Blueprint $table) {
```

```
    $table->string('role')->default('user');
```

```
});
```

```
...
```

```
**User Model Methods**:
```



```

```php
class User extends Authenticatable
{
    public function isAdmin(): bool
    {
        return $this->role === 'admin';
    }

    public function isModerator(): bool
    {
        return $this->role === 'moderator';
    }

    public function hasRole(string $role): bool
    {
        return $this->role === $role;
    }

    public function canModerate(): bool
    {
        return in_array($this->role, ['admin', 'moderator']);
    }
}
```

```

**\*\*Gates for Roles\*\*** (in AppServiceProvider):

```

```php
Gate::define('manage-users', function (User $user) {
    return $user->isAdmin();
});

Gate::define('moderate-posts', function (User $user) {
    return $user->canModerate();
});

Gate::define('edit-any-post', function (User $user) {
    return $user->canModerate();
});
```

```

**\*\*Usage\*\*:**

```

```php
// In Controller
if (auth()->user()->isAdmin()) {
    // Admin logic
}
```

```

```
// In Blade
@can('manage-users')
    Manage Users
@endcan

@if(auth()->user()->canModerate())
    

Moderate


@endif
```
```

---

## ## 🌀 What We Learned

### ### 1. Authentication Basics

- Laravel Breeze setup
- Login, Register, Logout
- Password reset
- Email verification

### ### 2. Authorization with Gates

- Defining Gates
- Checking permissions with `Gate::allows()` and `Gate::denies()`
- Using `Gate::authorize()` in controllers
- Using `@can` and `@cannot` in Blade

### ### 3. Policies

- Creating Policies
- Policy methods (viewAny, view, create, update, delete, etc.)
- Using `\$this->authorize()` in controllers
- Auto-discovery of Policies

### ### 4. Middleware

- Using `auth` middleware
- Using `guest` middleware
- Using `verified` middleware
- Creating custom middleware

### ### 5. API Authentication

- Laravel Sanctum setup
- Creating API tokens
- Protecting API routes
- Token abilities

### ### 6. Roles & Permissions

- Adding roles to users
- Role-based Gates

- Custom middleware for roles
- Role checking methods

---

## ## 🧪 Testing Your Implementation

### ### Test Authentication:

1. Visit `/register` and create an account
2. Visit `/login` and log in
3. Visit `/dashboard` (should see dashboard)
4. Logout and try to visit `/dashboard` (should redirect to login)

### ### Test Authorization:

1. Create a post (only logged-in users)
2. Try to edit own post (should work)
3. Try to edit another user's post (should get 403)
4. Try as admin (should work)

### ### Test API:

1. Use Postman or cURL to register via API
2. Login via API and get token
3. Use token to access protected routes
4. Test logout endpoint

---

## ## 📄 Common Issues & Solutions

### ### Issue 1: 403 Forbidden on all actions

**\*\*Solution\*\*:** Check your Policy's `before()` method - it might be blocking everything

### ### Issue 2: Middleware not working

**\*\*Solution\*\*:** Make sure middleware is registered in `bootstrap/app.php` or `app/Http/Kernel`

### ### Issue 3: Gate not found

**\*\*Solution\*\*:** Ensure Gates are defined in `AppServiceProvider::boot()` method

### ### Issue 4: Token authentication not working

**\*\*Solution\*\*:**

- Check Sanctum middleware is in `api` middleware group
- Ensure `HasApiTokens` trait is in User model
- Verify Bearer token format: `Authorization: Bearer {token}`

---

## ## 🔗 Next Steps

After completing these exercises:

1. Implement a complete blog system with authorization
2. Add more roles (editor, contributor)
3. Create a permission system
4. Implement API rate limiting
5. Add two-factor authentication

---

**\*\*Happy Coding!\*\***